

Backend — Conceptos Fundamentales

API REST · Contrato · OpenAPI · FastAPI · Fastify

Desarrollo de Software — 4° año Ing. de Software
UTN Facultad Regional La Plata

¿Qué es Backend?



Lógica del servidor

Procesa peticiones, ejecuta reglas de negocio, accede a datos. El usuario **no ve** esta capa.



API como interfaz

Expone funcionalidad a través de una **API** (Application Programming Interface). El frontend consume la API, no el backend directamente.



Persistencia + Seguridad

Maneja bases de datos, autenticación, autorización, sesiones, archivos, logs.

Analogía: Si una app web fuera un restaurante...

- **Frontend** = el menú y el mozo (lo que el cliente ve e interactúa)
- **Backend** = la cocina (donde procesan los ingredientes y preparan el plato)

API REST

¿Qué es REST?

REpresentational **St**ate **T**ransfer — estilo de arquitectura para diseñar APIs web. Cada recurso tiene una **URL**, y se opera con **verbos HTTP**.

GET	/usuarios	→ listar
POST	/usuarios	→ crear
GET	/usuarios/:id	→ obtener uno
PUT	/usuarios/:id	→ reemplazar
PATCH	/usuarios/:id	→ actualizar parcial
DELETE	/usuarios/:id	→ eliminar

Principios REST

Principio	¿Qué significa?
Stateless	Cada request tiene toda la info. No hay sesión en el server
Recursos	Todo es un recurso identificado por URL
Verbose HTTP	GET, POST, PUT, PATCH, DELETE tienen semántica definida
Respuesta estándar	Códigos de estado HTTP + body JSON

 **REST ≠ CRUD.** REST es un estilo arquitectónico. CRUD es un patrón de operaciones. Una API REST bien diseñada va más allá del CRUD.

Verbos HTTP y Códigos de Estado

Verbos

Verbo	Acción	¿Idempotente?	¿Seguro?
GET	Obtener recurso	✓	✓
POST	Crear recurso	✗	✗
PUT	Reemplazar recurso	✓	✗
PATCH	Actualizar parcial	✗	✗
DELETE	Eliminar recurso	✓	✗

Idempotente: múltiples requests idénticos producen el mismo resultado.

Seguro: no modifica el estado del servidor.

Códigos de Estado

Rango	Significado	Ejemplos
1xx	Informativo	101 Switching Protocols
2xx	Éxito	200 OK · 201 Created · 204 No Content
3xx	Redirección	301 Moved · 304 Not Modified
4xx	Error del cliente	400 Bad Request · 401 Unauthorized · 404 Not Found · 409 Conflict
5xx	Error del servidor	500 Internal · 503 Service Unavailable

Contrato de API

¿Qué es un contrato?

Un **acuerdo formal** entre el cliente (frontend, otra API, app mobile) y el servidor sobre **cómo se comunican**:

Cliente	Servidor
¿Qué URLs existen?	
¿Qué métodos aceptan?	
¿Qué datos necesito enviar?	
¿Qué forma tiene la respuesta?	
¿Qué códigos de error puede dar?	

Sin contrato → **inconsistencias, bugs, equipos pisándose.**

💡 ***El contrato NO es un documento PDF que escribís al inicio y se desactualiza. El contrato VIVE en el código y se genera automáticamente.***

Sin contrato vs Con contrato

Sin contrato 🚧	Con contrato ✅
"Che, ¿qué me devuelve este endpoint?"	Spec documentada
Frontend espera {name} y el backend devuelve {nombre}	Schemas compartidos
"Avisame cuando cambies algo"	Versionado semántico
Documentación desactualizada en un PDF	Documentación autogenerada

OpenAPI / Swagger

¿Qué es OpenAPI?

Un **estándar abierto** para describir APIs REST usando JSON o YAML. Antes se llamaba Swagger Specification.

```
openapi: 3.0.3
info:
  title: Mi API
  version: 1.0.0
paths:
  /usuarios:
    get:
      summary: Listar usuarios
      responses:
        '200':
          description: OK
          content:
            application/json:
              schema:
                type: array
```

Herramientas del ecosistema

Herramienta	¿Para qué?
Swagger UI	Interfaz web interactiva para probar endpoints
Swagger Editor	Editor online con preview en vivo
OpenAPI Generator	Genera clients HTTP en 50+ lenguajes
Swagger Codegen	Genera server stubs
Postman / Insomnia	Importan specs OpenAPI

En este curso: No escribimos OpenAPI a mano. **Fastify y FastAPI lo generan solos** a partir de schemas.

Swagger UI en acción

Lo que genera automáticamente

Schema de ruta
↓
@fastify/swagger / fastapi.FastAPI
↓
Spec OpenAPI 3.0 (JSON)
↓
Swagger UI (interfaz interactiva)

Beneficios concretos

-  **Documentación viva** — se genera del código
-  **Try it out** — probá endpoints desde el navegador
-  **Interoperable** — cualquier herramienta consume OpenAPI
-  **Autodescubrible** — entendés toda la API en 5 min

Ejemplo: `GET /health` con schema → Swagger genera:

```
"responses": { "200": { "content": {  
  "application/json": { "schema": {  
    "type": "object",  
    "properties": {  
      "status": { "type": "string" },  
      "service": { "type": "string" }    }  
  }  
}}
```



FastAPI

¿Qué es?

Framework web moderno para Python (3.8+) basado en **type hints** y **Pydantic**. Creado por Sebastián Ramírez en 2018.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Mi API")

class Usuario(BaseModel):
    nombre: str
    edad: int

@app.post("/usuarios")
def crear_usuario(usuario: Usuario):
    return {"id": 1, **usuario.model_dump()}
```

Stack tecnológico

FastAPI

- Starlette (web)
- Pydantic (validación/schemas)
- Uvicorn (servidor ASGI)
- OpenAPI autogenerado

Características clave

- ✓ Validación automática con type hints
- ✓ Async nativo
- ✓ Documentación Swagger automática
- ✓ Tipado completo (editor autocompletado)
- ✓ Rendimiento excelente (a la par de Node.js/Go)

FastAPI — Código en acción

```
from fastapi import FastAPI
from pydantic import BaseModel

class Usuario(BaseModel):
    nombre: str
    email: str
    edad: int

app = FastAPI(title="Usuarios API", version="0.1.0")

@app.get("/")
def raiz():
    return {"message": "Hola desde FastAPI"}

@app.get("/health")
def health():
    return {"status": "ok", "service": "fastapi-hello"}

@app.post("/usuarios", status_code=201)
def crear_usuario(usuario: Usuario):
    return {"id": 1, **usuario.model_dump()}
```

Lo que pasa automáticamente sin escribir más código:



¿Qué es?

Framework web para Node.js enfocado en **rendimiento** y **baja sobrecarga**. Creado por Matteo Collina y Tomas Della Vedova en 2017.

```
import Fastify from "fastify";

const app = Fastify({ logger: true });

app.get("/usuarios/:id", async (request) => {
  const { id } = request.params;
  return { id, nombre: "Ejemplo" };
});

await app.listen({ port: 3000 });
```

Stack tecnológico

Fastify

- find-my-way (router radix tree)
- fast-json-stringify (serialización)
- avvio (plugin bootstrapper)
- Pino (logger estructurado)
- @fastify/swagger (OpenAPI)

Características clave

- ⚡ 2-3x más rápido que Express
- 📄 Schema-first (JSON Schema)
- 🔌 Sistema de plugins encapsulados
- 📝 Logger nativo (Pino)
- 📄 Swagger autogenerado con schemas

Fastify — Código en acción

```
import Fastify from "fastify";
import fastifySwagger from "@fastify/swagger";
import fastifySwaggerUi from "@fastify/swagger-ui";

const app = Fastify({ logger: true });

await app.register(fastifySwagger, {
  openapi: { info: { title: "Mi API", version: "0.1.0" } },
});
await app.register(fastifySwaggerUi, { routePrefix: "/docs" });

app.get("/", {
  schema: { summary: "Raíz", response: { 200: {
    type: "object", properties: { message: { type: "string" } },
  } } },
}, async () => ({ message: "Hola desde Fastify" }));

app.get("/health", {
  schema: { summary: "Health check", response: { 200: {
    type: "object", properties: {
      status: { type: "string" }, service: { type: "string" },
    },
  } } },
}, async () => ({ status: "ok", service: "fastify-hello" }));

await app.listen({ port: 3000 });
```

FastAPI vs Fastify

Aspecto	FastAPI 🐍	Fastify ⚡
Lenguaje	Python 3.8+	TypeScript / Node.js
Server	Uvicorn (ASGI)	Node.js HTTP
Schemas	Pydantic (type hints)	JSON Schema
Validación	Automática por tipo	Con schema JSON
Serializer	Pydantic <code>.model_dump()</code>	<code>fast-json-stringify</code>
Async	<code>async def</code> nativo	Siempre async
Swagger	<code>/docs</code> automático (sin plugins)	<code>@fastify/swagger</code> + <code>@fastify/swagger-ui</code>
Rendimiento	Muy alto (~25k req/s)	Muy alto (~35k req/s)
Ecosistema	Starlette + Pydantic	Plugin encapsulado
Curva de aprendizaje	Baja (Python puro)	Media (TypeScript + plugins)

¿Cuándo usar cuál?

🐍 **FastAPI** Si el equipo viene de Python, necesitas prototipar rápido, o tu ecosistema incluye ML/data

El contrato en acción

FastAPI — Schema como modelo

```
class Usuario(BaseModel):
    nombre: str
    email: str
    edad: int

@app.post("/usuarios")
def crear(usuario: Usuario):
    ...
```

El **modelo Pydantic** ES el contrato.
Swagger lo refleja automáticamente.

- ✓ Sin duplicación
- ✓ Un cambio, un lugar
- ✓ Validación + doc + spec

Fastify — Schema como objeto

```
const schema = {
  body: {
    type: "object",
    required: ["nombre", "email"],
    properties: {
      nombre: { type: "string" },
      email: { type: "string", format: "email" },
      edad: { type: "integer" },
    },
  },
};

app.post("/usuarios", { schema }, handler);
```

El **schema JSON** ES el contrato.
Swagger lo refleja automáticamente.

- ✓ Sin duplicación
- ✓ Un cambio, un lugar
- ✓ Validación + doc + spec

Resumen

Conceptos

 **Backend** = lógica del servidor, expuesta vía API

 **REST** = estilo arquitectónico con verbos HTTP y recursos

 **Contrato** = acuerdo formal entre cliente y servidor

 **OpenAPI** = estándar para describir APIs

Frameworks

 **FastAPI** Python, type hints, Pydantic, async nativo

 **Fastify** TypeScript, schema-first, plugins encapsulados

El principio fundamental

El schema es el contrato.

El contrato genera la documentación.

La documentación es la API.

Referencias

Recurso	URL
FastAPI	https://fastapi.tiangolo.com/
Fastify	https://fastify.dev/
OpenAPI Specification	https://spec.openapis.org/oas/v3.0.3
Swagger UI	https://swagger.io/tools/swagger-ui/
JSON Schema	https://json-schema.org/
Pydantic	https://docs.pydantic.dev/
RESTful API Design	https://restfulapi.net/

Presentación generada con **Marp** · Markdown Presentation Ecosystem